



UNIVERSIDAD  
DE GRANADA



# Técnicas de los Sistemas Inteligentes

Grado en Ingeniería Informática

**Curso 2025-26. Seminario 3**

**Planificación Clásica (PDDL) y Práctica 3**

Pablo Mesejo Santiago y David Criado Ramón

**Departamento de Ciencias de la  
Computación e Inteligencia Artificial**

<http://decsai.ugr.es>

Stuart Russell and Peter Norvig, *Artificial Intelligence: A modern Approach*, 4ª Edición, Global Edition, Ed. Pearson, 2022.

Capítulo 11. Y, si se quiere profundizar más en problemas de decisión secuencial, Capítulo 16.

Malik Ghallab, Dana Nau, and Paolo Traverso. *Automated Planning and Acting*. Cambridge University Press, 2016.

<https://www.cs.umd.edu/~nau/planning/slides/>

Hector Geffner and Blai Bonet. *A Concise Introduction to Models and Methods for Automated Planning*. Morgan & Claypool Publishers, 2013.

# Índice

1. ¿Qué es la Planificación?
  - i. Problema de ejemplo
  - ii. Acciones
  - iii. Representación del mundo
2. Planning Domain Definition Language (PDDL)
  - i. Ontología PDDL
  - ii. Acciones PDDL
  - iii. PDDL avanzado
3. Planificador Fast Downward
4. Otras consideraciones
5. Práctica 3
6. Preguntas frecuentes



# Índice

1. **¿Qué es la Planificación?**
  - i. Problema de ejemplo
  - ii. Acciones
  - iii. Representación del mundo
2. Planning Domain Definition Language (PDDL)
  - i. Ontología PDDL
  - ii. Acciones PDDL
  - iii. PDDL avanzado
3. Planificador Fast Downward
4. Otras consideraciones
5. Práctica 3
6. Preguntas frecuentes

# ¿Qué es la Planificación?

En las prácticas anteriores hemos visto técnicas para que un agente resuelva un problema simple, ajustando su ejecución a ciertas restricciones de bajo nivel.

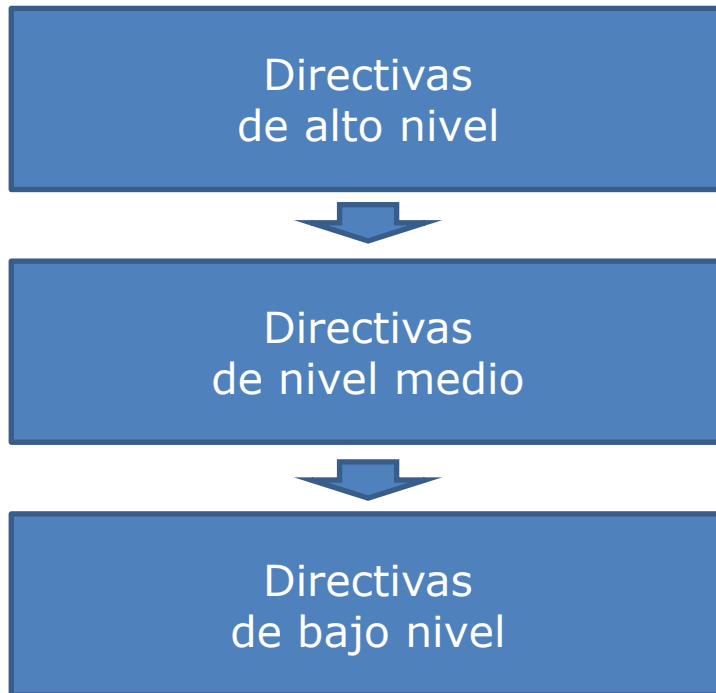
Pero...

Estas técnicas no son suficientes para **resolver problemas complejos** que requieran **sincronizar distintos tipos de subtareas o agentes**, o que necesiten **razonar sobre acciones que producen un efecto sobre el mundo**.



# ¿Qué es la Planificación?

## Planificación automática y agentes:



Determinar a alto nivel qué acciones deben ejecutarse, y en qué orden, para alcanzar un objetivo propuesto **(Planificación automática)**

Desarrollar un plan de acción para ejecutar una tarea concreta **(Técnicas Deliberativas de la Práctica 1)**

Ajustar el plan de acción a las restricciones del mundo/problema para ejecutarlo correctamente **(Técnicas de la Práctica 2 + Técnicas Reactivas)**



# ¿Qué es la Planificación?

La planificación automática es una de las áreas clásicas dentro de la Inteligencia Artificial.

**Artificial Intelligence: A Modern Approach, 4th US ed.**

by Stuart Russell and Peter Norvig

The authoritative, most-used AI textbook, adopted by over 1500 schools.

Table of Contents for the US Edition (or see the Global Edition)

Preface (pdf); Contents with subsections

## I Artificial Intelligence

- 1 Introduction ... 1
- 2 Intelligent Agents ... 36

## II Problem-solving

- 3 Solving Problems by Searching ... 63
- 4 Search in Complex Environments ... 110
- 5 Adversarial Search and Games ... 146
- 6 Constraint Satisfaction Problems ... 180

## III Knowledge, reasoning, and planning

- 7 Logical Agents ... 208
- 8 First-Order Logic ... 251
- 9 Inference in First-Order Logic ... 280
- 10 Knowledge Representation ... 314
- 11 Automated Planning ... 344

## IV Uncertain knowledge and reasoning

- 12 Quantifying Uncertainty ... 385
- 13 Probabilistic Reasoning ... 412
- 14 Probabilistic Reasoning over Time ... 461
- 15 Probabilistic Programming ... 500
- 16 Making Simple Decisions ... 528
- 17 Making Complex Decisions ... 562
- 18 Multiagent Decision Making ... 599

## V Machine Learning

- 19 Learning from Examples ... 651
- 20 Learning Probabilistic Models ... 721
- 21 Deep Learning ... 750
- 22 Reinforcement Learning ... 789

## VI Communicating, perceiving, and acting

- 23 Natural Language Processing ... 823
- 24 Deep Learning for Natural Language Processing ... 856
- 25 Computer Vision ... 881
- 26 Robotics ... 925

## VII Conclusions

- 27 Philosophy, Ethics, and Safety of AI ... 981
- 28 The Future of AI ... 1012
- Appendix A: Mathematical Background ... 1023
- Appendix B: Notes on Languages and Algorithms ... 1030
- Bibliography ... 1033 (pdf and LaTeX .bib file and bib data)
- Index ... 1069 (pdf)

Exercises (website)

Figures (pdf)

Code (website); Pseudocode (pdf)

Covers: US, Global

Referencia recomendable:

<https://projects.laas.fr/planning/>

## AUTOMATED PLANNING AND ACTING

Malik Ghallab  
LAAS-CNRS  
Toulouse, France

Dana Nau  
University of Maryland  
College Park, MD, USA

Paolo Traverso  
FBK ICT-IRST  
Trento, Italy

Relacionada con *Reinforcement Learning* y *Sequential Decision Making*.

# ¿Qué es la Planificación?

Las técnicas de planificación requieren dos elementos:

1. **Dominio:** Un conjunto de acciones (y sus efectos esperados).
2. **Problema:** Una definición del estado inicial y objetivo.

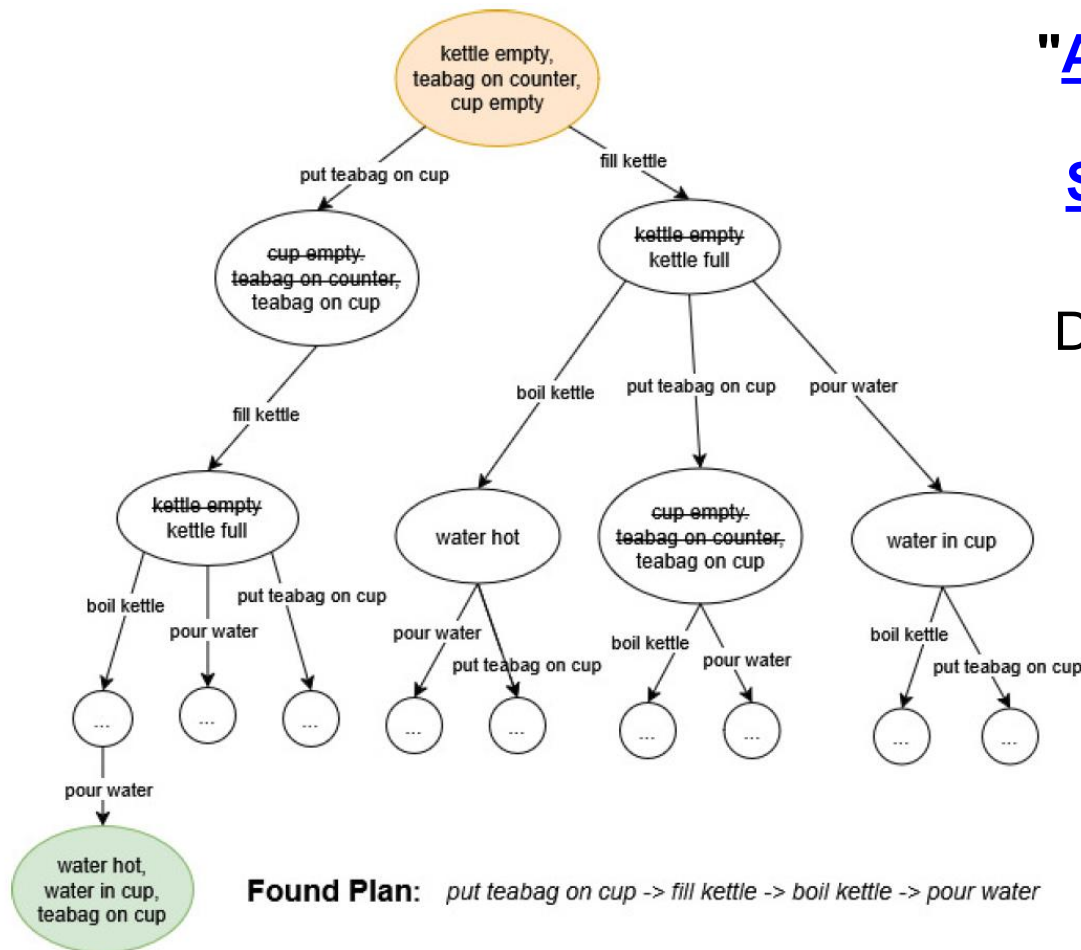


Partiendo de una representación del mundo (**estado inicial**), un **planificador proporciona una lista de acciones** que, al ser aplicadas en orden, generan una representación del mundo deseada (**estado objetivo**).





# ¿Qué es la Planificación?



## "Application of Neuro-Symbolic Artificial Intelligence to Sequential Decision Making"

Carlos Núñez Molina

Defensa de la Tesis: 10/02/2025

Universidad de Granada

## "Neurosymbolic AI for Automated Planning – Integrating LLMs with Classical and Hierarchical Planning"

Israel Puerta Merino

# Índice

1. ¿Qué es la Planificación?
  - i. **Problema de ejemplo**
  - ii. Acciones
  - iii. Representación del mundo
2. Planning Domain Definition Language (PDDL)
  - i. Ontología PDDL
  - ii. Acciones PDDL
  - iii. PDDL avanzado
3. Planificador Fast Downward
4. Otras consideraciones
5. Práctica 3
6. Preguntas frecuentes

## Problema de ejemplo: El mono y los plátanos

- Un mono en un laboratorio tiene lejos de su alcance un racimo de plátanos.
- Una caja permite alcanzar los plátanos si éste se sube en ella.
- El mono está en una posición desde la que no alcanza los plátanos. Los plátanos y la caja están en posiciones distintas.
- El mono puede **desplazarse** de una ubicación a otra, **empujar** la caja, **subir** a la caja, **coger** los plátanos y **bajar** de la caja.



[https://en.wikipedia.org/wiki/Monkey\\_and\\_banana\\_problem](https://en.wikipedia.org/wiki/Monkey_and_banana_problem)

# Índice

1. ¿Qué es la Planificación?
  - i. Problema de ejemplo
  - ii. Acciones**
  - iii. Representación del mundo
2. Planning Domain Definition Language (PDDL)
  - i. Ontología PDDL
  - ii. Acciones PDDL
  - iii. PDDL avanzado
3. Planificador Fast Downward
4. Otras consideraciones
5. Práctica 3
6. Preguntas frecuentes



# Acciones tipo STRIPS

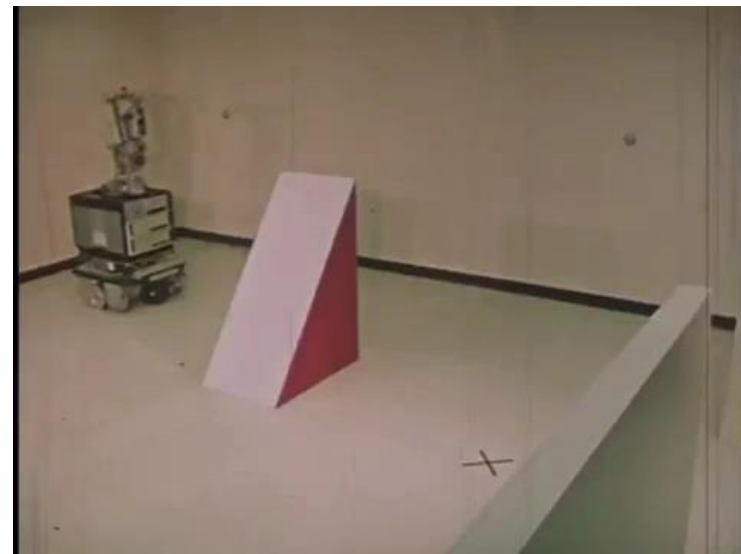
**Acción** cogerPlátanos

Parámetros(.....)

Precondiciones(.....)

Postcondiciones(.....)

- Cabecera
  - Nombre de la acción y parámetros.
- Precondiciones (Requisitos)
  - Condiciones que deben cumplirse para poder ejecutar la acción.
- Postcondiciones (Efectos)
  - Cómo modifica el mundo la acción.
 Definidos como:
  - Una lista de cosas que añadir al mundo.
  - Una lista de cosas que eliminar del mundo.



[Shakey: Experiments in Robot Planning and Learning \(1972\)](#)

*Shakey The Robot* (1966-1972). DARPA project developed at SRI International. 1st general-purpose mobile robot.

[Story of A\\* and STRIPS by Nils J. Nilsson](#)

Stanford Research Institute Problem Solver (**STRIPS**) es el lenguaje formal usado para codificar las acciones.

- Richard E. Fikes and Nils J. Nilsson. "STRIPS: A new approach to the application of theorem proving to problem solving." *Artificial Intelligence* 2.3-4 (1971): 189-208.

# El problema del marco

- Cada acción se aplica sobre un estado previo del mundo (pre-estado) y genera un estado sucesor (post-estado)
- ¿Qué aspectos del mundo cambian cuando la ejecutamos?
  - Esto es conocido como el [Problema del Marco](#):
    - **¿Qué es lo que se mantiene (marco) y qué es lo que cambia en el mundo tras llevar a cabo una acción?**
- Se trata de un problema “irresoluble”, en el sentido de que en el mundo real no podemos saber con absoluta seguridad cuáles son todos los efectos de una acción.
- STRIPS intenta resolver el problema del marco a través de la [Hipótesis del Mundo Cerrado](#):
  - **Si no se sabe si una acción ha cambiado una cierta característica del dominio, se asume que no la ha cambiado.**

John McCarthy and Patrick J. Hayes. "Some philosophical problems from the standpoint of artificial intelligence." *Machine Intelligence* 4 (1969): 463-502.



# Índice

1. ¿Qué es la Planificación?
  - i. Problema de ejemplo
  - ii. Acciones
  - iii. **Representación del mundo**
2. Planning Domain Definition Language (PDDL)
  - i. Ontología PDDL
  - ii. Acciones PDDL
  - iii. PDDL avanzado
3. Planificador Fast Downward
4. Otras consideraciones
5. Práctica 3
6. Preguntas frecuentes



# Representación del mundo

- La representación del mundo (**Ontología**) es una descripción formal de los objetos y relaciones entre objetos del mundo.
- 1. **Objetos:** los elementos del mundo.
  - Representan a los elementos del mundo
  - Pueden tener un tipo (mono, caja)
- 2. **Relaciones:** indican propiedades de los objetos.
  - Estas propiedades pueden ser intra-objeto (el **mono** tiene plátanos) o inter-objeto (el **mono** está encima de la **caja**)
  - Se representan usando lógica de primer orden (predicados)
    - tienePlátanos(**mono**)
    - sobre(**mono**, **caja**)

Función que devuelve un valor Booleano

# Representación del mundo

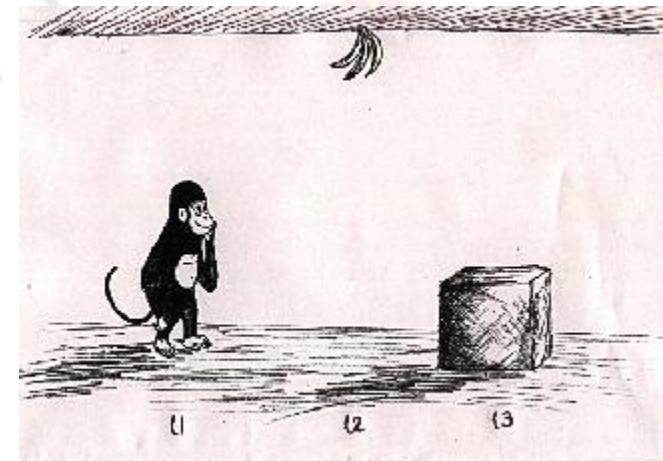
- **Estado:** Representación del mundo en un momento concreto. Se define como una conjunción de predicados lógicos junto con una colección de objetos.

- **Objetos:**

- Mono1 – Mono
- Caja1 – Caja
- Localización1 – Localización
- Localización2 – Localización
- Localización3 – Localización

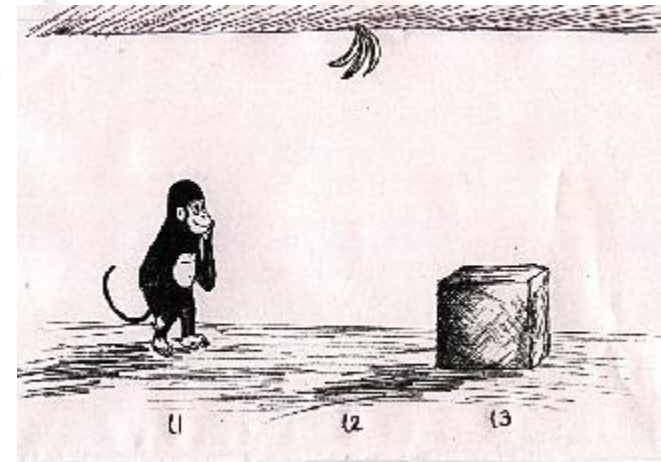
Identificador  
del objeto

Tipo del objeto



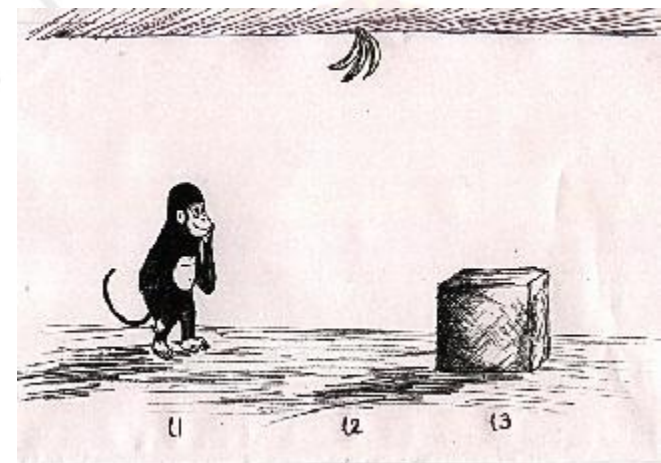
# Representación del mundo

- **Estado:** Representación del mundo en un momento concreto. Se define como una conjunción de predicados lógicos junto con una colección de objetos.
- **Relaciones:**
  - En(Mono1, Localización1)
  - En(Caja1, Localización3)
  - PlatanoEn(Localización2)



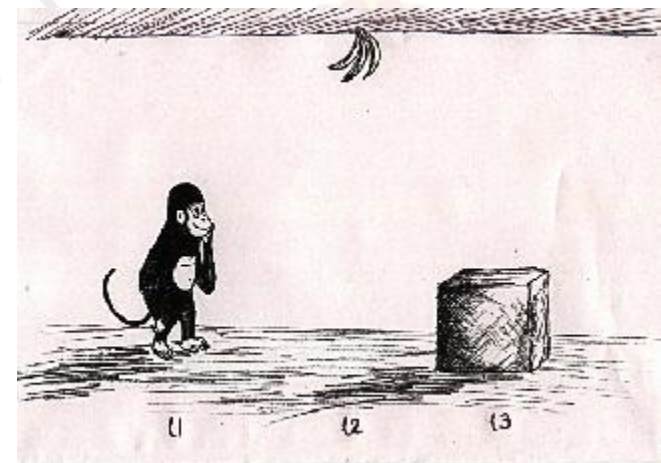
# Representación del mundo

- Los planificadores, como decíamos, suelen seguir la **Hipótesis del Mundo Cerrado (CWA)**
  - “Si no conozco algo lo considero falso.”
- Esto evita escribir todas las relaciones falsas
  - $\neg \text{En}(\text{Mono1}, \text{Localización2})$
  - $\neg \text{En}(\text{Mono1}, \text{Localización3})$
  - $\neg \text{tienePlátano}(\text{Mono1})$
  - ...



# Representación del mundo

- **Estado Objetivo:** Representación del mundo al que queremos llegar.
  - No se requiere definir el estado completo. Solo el pequeño subconjunto de predicados que queremos que sean ciertos al finalizar el plan.
- tienePlátano(Mono1)



# Índice

1. ¿Qué es la Planificación?
  - i. Problema de ejemplo
  - ii. Acciones
  - iii. Representación del mundo
2. **Planning Domain Definition Language (PDDL)**
  - i. **Ontología PDDL**
  - ii. **Acciones PDDL**
  - iii. **PDDL avanzado**
3. Planificador Fast Downward
4. Otras consideraciones
5. Práctica 3
6. Preguntas frecuentes





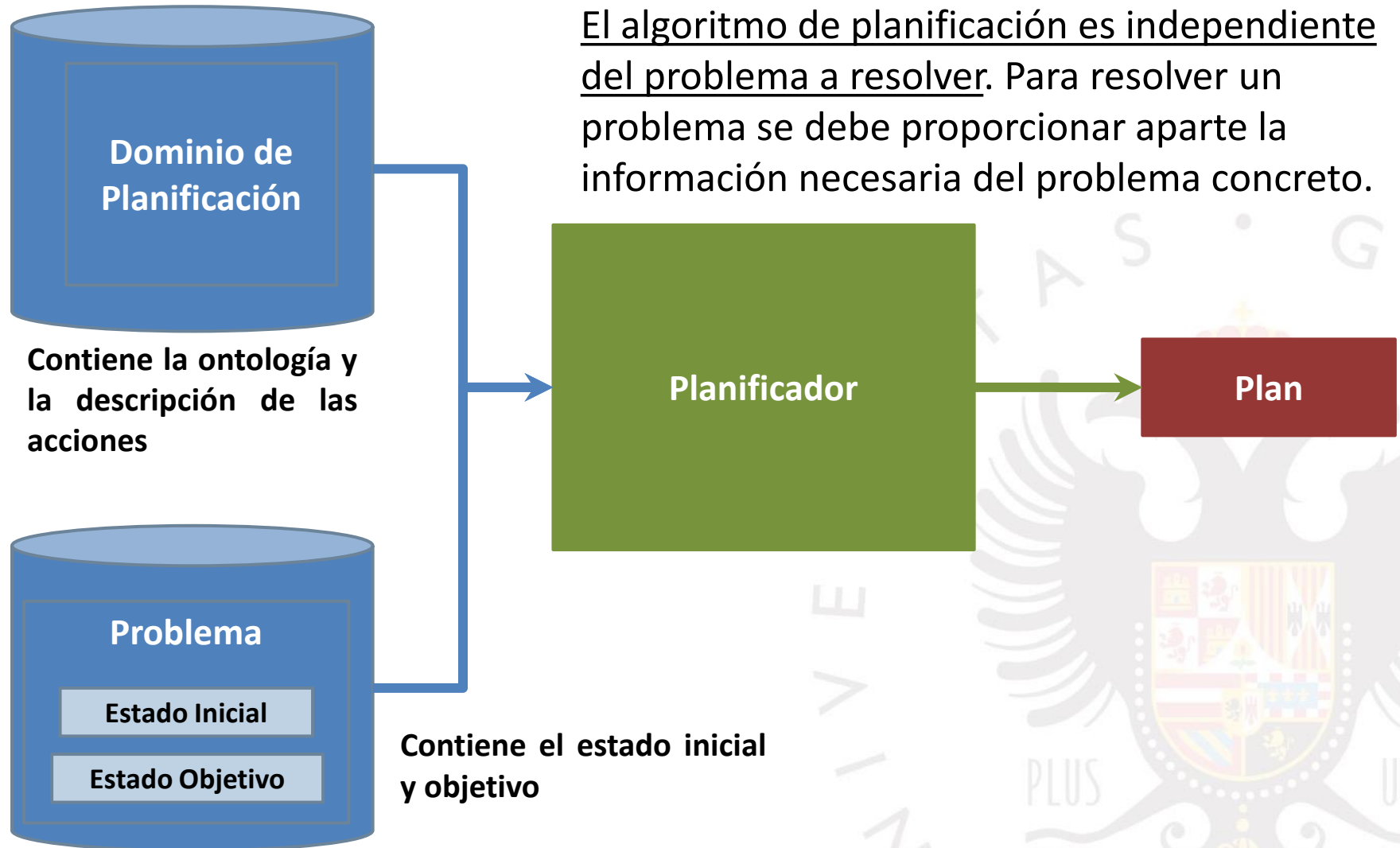
# PDDL

- PDDL
  - Drew McDermott et al. "PDDL-the planning domain definition language." (1998).
- Lenguaje estándar para la representación de dominios de planificación clásicos. Inspirado en STRIPS y basado en LISP.
  - “[\*Lots of Irritating Superfluous Parentheses\*](#)”
- En su versión clásica, todas las propiedades y relaciones entre los objetos se conocen inicialmente o pueden conocerse durante el proceso de planificación (Conocimiento Completo)
  - Sistema completamente determinista, observable y estático.
  - Número finito de estados.
  - Los efectos de las acciones son conocidos a priori.
  - Cambios en el mundo exclusivamente producidos por la ejecución de las acciones.
  - Los objetivos también son estáticos y no cambian.

# PDDL

- Material online adicional sobre PDDL
  - Writing Planning Domains and Problems (breve introducción)  
<http://users.cecs.anu.edu.au/~patrik/pddlman/writing.html>
  - Online PDDL Reference  
<https://planning.wiki/ref/pddl>
  - Otros recursos didácticos sobre PDDL y planificación  
<http://education.planning.domains/>  
<https://planning.wiki/>

# Planificación Independiente de Dominio



# Dominio PDDL

```

(define (domain <domain name>)
  (:requirements <requirements spec>)
  (:types <types spec>)
  (:predicates <predicates spec>)
  (:action <action spec>)
  (:action <action spec>)
  ...
)
```

Un dominio tiene las siguientes partes:

- cabecera
- requisitos
- tipos
- predicados
- acciones

**Nota:** los comentarios, en PDDL, se introducen con ;

# Dominio PDDL

```
(define (domain <domain name>)
  (:requirements <requirements spec>)
  (:types <types spec>)
  (:predicates <predicates spec>)
  (:action <action spec>)
  (:action <action spec>)
  ...
)
```

Cabecera que indica el nombre del dominio.

Debe concordar con el nombre del dominio que aparece en el fichero del problema (que se verá más adelante). De lo contrario:

**"The domain name specified by the task (tsidominio15) does not match the name specified by the domain file (tsidominio5)."**

# Dominio PDDL

```
(define (domain <domain name>)
  (:requirements <requirements spec>)
  (:types <types spec>)
  (:predicates <predicates spec>)
  (:action <action spec>)
  (:action <action spec>)
  ...
)
```

Requisitos y capacidades que debe cumplir el planificador para resolver problemas con el dominio. Amplían o modifican el comportamiento estándar de PDDL.

```
(:requirements
  :req1
  :req2
  :req3
)
```

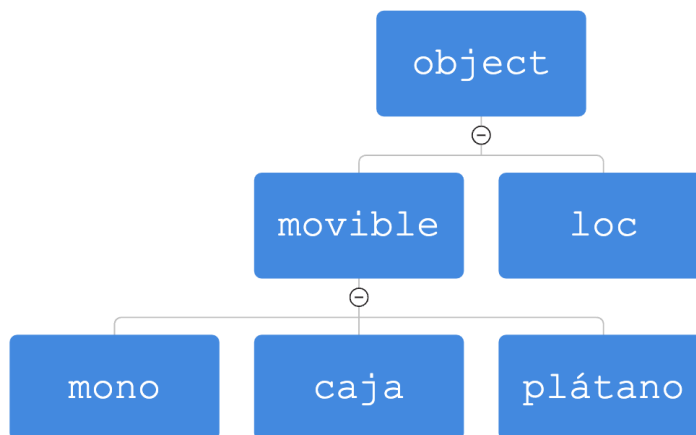
<https://planning.wiki/ref/pddl/requirements>

**Nota:** como norma general, no es un elemento muy crítico, dado que los *planners* cargan, por defecto, buena parte de las funcionalidades.



# Dominio PDDL

```
(define (domain <domain name>)
  (:requirements <requirements spec>)
  (:types <types spec>)
  (:predicates <predicates spec>)
  (:action <action spec>)
  (:action <action spec>)
  ...
)
```



Tipos de los objetos del dominio

- Los tipos pueden depender de un supertipo

```
(:requirements
  :typing
)
(:types
  movible loc - object
  mono caja plátano - movible
)
```

# Dominio PDDL

```

(define (domain <domain name>)
  (:requirements <requirements spec>)
  (:types <types spec>)
  (:predicates <predicates spec>)
  (:action <action spec>)
  (:action <action spec>)
  ...
)

```

Predicados de la ontología

“El objeto x de tipo movable está en la localización y”

“El mono x tiene el plátano”

“El mono x está sobre la caja y”

“El plátano está en la localización x”

```

(:predicates
  (en ?x - movable ?y - loc)
  (tienePlátano ?x - mono)
  (sobre ?x - mono ?y - caja)
  (plátanoEn ?x - loc)
)

```

Existe el comando `either` para indicar que un objeto puede ser de varios tipos.

Ejemplo:

```

(necesita ?tipo - (either tipoEdificio tipoUnidad) ?rec - Recurso)

```

# Dominio PDDL

```
(define (domain <domain name>)
  (:requirements <requirements spec>)
  (:types <types spec>)
  (:predicates <predicates spec>)
  (:action <action spec>)
  (:action <action spec>)
  ...
)
```

Lista de acciones del dominio.

Las vemos en mayor detalle en las siguientes diapositivas.

# Dominio PDDL: Acciones

- Acciones:
  - Requirement **:strips**

```

(:action irA
  :parameters (?m - mono ?x ?y - localizacion)
  :precondition
    (and
      (en ?m ?x)
    )
  :effect
    (and
      (en ?m ?y)
      (not (en ?m ?x))
    )
)
    
```

Este *and* solo es imprescindible si hay más de un predicado implicado.

Esta acción, llamada *irA*, necesita un objeto *m* de tipo *mono*, y dos localizaciones (*x*, *y*)

Para que el mono *m* pueda desplazarse desde *x* (de tipo *localización*), debe estar en *x* inicialmente.

Como consecuencia de ejecutar la acción, el mono *m* pasa de estar en *x* a estar en *y*.

# Dominio PDDL: Acciones

- Cabecera:

```
(:action irA
  :parameters (?m - mono ?x ?y - localizacion)
  :precondition
    (and
      (en ?m ?x)
    )
  :effect
    (and
      (en ?m ?y)
      (not (en ?m ?x))
    )
)
```

- Contiene un nombre de la acción (irA) y una lista de parámetros (?m, ?x, ?y) junto con sus tipos.

# Dominio PDDL: Acciones

- Precondiciones:

```

(:action irA
  :parameters (?m - mono ?x ?y - localización)
  :precondition
    (and
      (en ?m ?x)
    )
  :effect
    (and
      (en ?m ?y)
      (not (en ?m ?x))
    )
)

```

- Las precondiciones se representan como una conjunción de predicados del mundo.

$$P1 \text{ Y } P2 \text{ Y } P3 \text{ Y } \dots \text{ Y } Pn$$

# Dominio PDDL: Acciones

- Efectos:

```
(:action irA
  :parameters (?m - mono ?x ?y - localización)
  :precondition
    (and
      (en ?m ?x)
    )
  :effect
    (and
      (en ?m ?y)
      (not (en ?m ?x))
    )
)
```

- Recordemos. Los efectos se separan en dos tipos:
  - Lista de predicados a añadir al estado previo.
  - Lista de predicados a borrar (se eliminan con **not**)



# Dominio PDDL

```
(define (domain mono)
  (:requirements :strips :typing)
  (:types
    movable localizacion - object
    mono caja - movable
  )
  (:predicates
    (en ?obj - movable ?x - localizacion)
    (tienePlatano ?m - mono)
    (sobre ?m - mono ?c - caja)
    (platanoEn ?x - localizacion)
  )

  (:action cogerPlatanos
    :parameters (?m - mono ?c - caja)
    :precondition
      (and
        (sobre ?m ?c)
      )
    :effect
      (and
        (tienePlatano ?m)
      )
  )
)
```

¿Esta acción cogerPlatanos presenta alguna limitación o inconsistencia semántica? ¿Está incompleta de algún modo?



**Nota:** Los primeros pasos para trabajar con PDDL que os recomendaremos más adelante, incluirán trabajar este dominio para completarlo y hacerlo más funcional.

# Fichero de problema

```
(define (problem <problem id>)
  (:domain <domain name>)
  (:objects <object spec>)
  (:init <initial facts spec>)
  (:goal <goal spec>)
)
```

Un fichero de problema tiene las siguientes partes:

- cabecera
- nombre del dominio al que pertenece
- los objetos
- el estado inicial
- el objetivo

# Fichero de problema

```
(define (problem <problem id>)
  (:domain <domain name>)
  (:objects <object spec>)
  (:init <initial facts spec>)
  (:goal <goal spec>)
)
```

Cabecera del problema y nombre

# Fichero de problema

```
(define (problem <problem id>)
```

```
  (:domain <domain name>)
```

```
  (:objects <object spec>)
```

```
  (:init <initial facts spec>)
```

```
  (:goal <goal spec>)
```

```
)
```

Nombre del dominio asociado al problema.

Este nombre debe ser igual al nombre definido en la cabecera del fichero de dominio.

# Fichero de problema

```
(define (problem <problem id>)
  (:domain <domain name>)
  (:objects <object spec>)
  (:init <initial facts spec>)
  (:goal <goal spec>)
)
```

Nombre y tipo de los objetos  
del problema

```
(:objects
  mono1 - mono
  caja1 - caja
  platano1 - platano
  loc1 loc2 loc3 - loc
)
```

# Fichero de problema

```
(define (problem <problem id>)
  (:domain <domain name>)
  (:objects <object spec>)
  (:init <initial facts spec>)
  (:goal <goal spec>)
)
```

Estado Inicial

En el *:init*, aunque haya múltiples predicados, no hace falta un *and*.

```
(:init
  (en mono1 loc1)
  (platanosEn loc2)
  (en caja1 loc3)
)
```

Recordad que PDDL sigue el CWA (*Closed-World Assumption*): todo lo que no se especifique en el *:init* se considera falso.

# Fichero de problema

```
(define (problem <problem id>)
  (:domain <domain name>)
  (:objects <object spec>)
  (:init <initial facts spec>)
  (:goal <goal spec>)
)
```

Objetivo a alcanzar

```
(:goal
  (and
    (tienePlátano mono1)
  )
)
```



# Problema PDDL

```
(define (problem monosp1)
  (:domain mono)
  (:objects
    mono1 - mono
    caja1 - caja
    localizacion1 localizacion2 localizacion3 - localizacion
  )
  (:init
    (en caja1 localizacion2)
    (platanoEn localizacion2)

    (sobre mono1 caja1)
  )
  (:goal
    (and
      (tienePlatano mono1)
    )
  )
)
```

## A modo de resumen

### **Fichero de Dominio**

- Define las reglas del mundo en que operamos:
  - jerarquía de tipos,
  - predicados y
  - acciones.

### **Fichero de Problema**

- Define la tarea concreta a resolver dentro del dominio:
  - objetos existentes en el mundo,
  - inicialización de los mismos, y
  - objetivo/estado a alcanzar.

## Consejos sobre PDDL

A la hora de escribir una acción hay que pensar que:

- Todo lo que subyace al proceso de planificación, al igual que en razonamiento con restricciones, es un **problema de búsqueda**.
  - El *planner* va a intentar cumplir con el objetivo dado, por medio de ir expandiendo acciones cuyas precondiciones sean ciertas.

Como consecuencia:

- Debemos diseñar una jerarquía de tipos adecuada y, en un principio, cuando definimos las acciones:
  - Las precondiciones deben restringir los predicados necesarios.
  - Los efectos deben modificar los predicados imprescindibles.
- No obstante....
  - Al igual que con CSP/COP, introducir cierta redundancia en las precondiciones puede ayudar a reducir el espacio de búsqueda (puede permitir reducir el número de acciones que se pueden ejecutar, es decir, reducir el factor de ramificación).
  - **Acotar adecuadamente las precondiciones de las acciones puede producir una importante aceleración de los tiempos de ejecución (p.ej. x100)**

# PDDL Avanzado (I)

## Precondiciones con disyunciones

ADL es un super-requerimiento que añade varios, entre otros *:disjunctive-preconditions*

Algunos requerimientos ya vienen incorporados por defecto, y no es necesario incluirlos.

- Requirement **:adl**
- La expresión lógica de la precondición de una acción no se limita a una conjunción (and), además puede incluir el operador de disyunción (or) y el anidado de expresiones

$$P1 \text{ Y } (P2 \text{ O } (P3 \text{ Y } P4)) \text{ Y } \dots \text{ Y } Pn$$

## PDDL Avanzado (II)

### Precondiciones con cuantificadores lógicos

- Permiten extender precondiciones y comprobar que una expresión lógica se cumple para un conjunto de objetos.
  - **forall** (todos deben cumplirla)
  - **exists** (al menos uno debe cumplirla)

```
(forall (?x - T)
  (expresión lógica)
)
```

“(expresión lógica) debe ser verdadera para cada objeto de tipo T.”

```
(exists (?x - T)
  (expresión lógica)
)
```

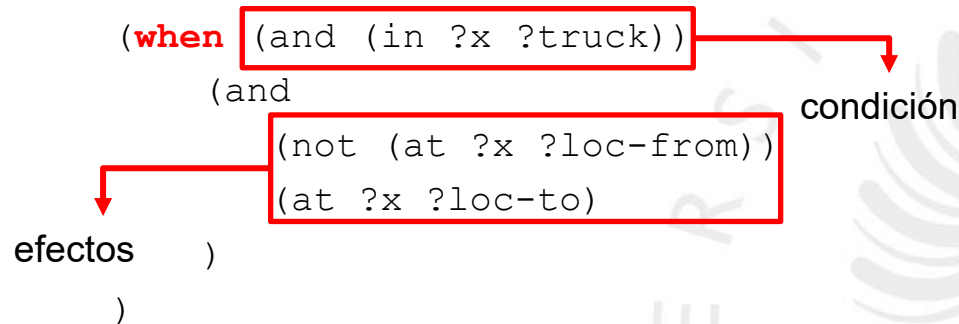
“(expresión lógica) debe ser verdadera para algún objeto de tipo T.”

- **forall/exists** pueden llegar a ser operaciones costosas.
  - De cara a la eficiencia, en ciertas ocasiones, puede ser conveniente incluir los objetos como parámetros de las acciones o declarar predicados auxiliares que las sustituyan.

# PDDL Avanzado (III)

## Efectos condicionales

- Efectos que se activan si es cierta alguna condición en el estado.



Si la condición es cierta, se ejecuta un bloque de efectos.

En el ejemplo ([https://www.cs.toronto.edu/~sheila/384/w11/Assignments/A3/veloso-PDDL\\_by\\_Example.pdf](https://www.cs.toronto.edu/~sheila/384/w11/Assignments/A3/veloso-PDDL_by_Example.pdf)): “Si un objeto x está dentro del camión [y el camión ha llegado a su destino], entonces dicho objeto x no está en la localización de origen sino en la de destino”

# PDDL Avanzado (IV)

## Efectos condicionales

- Puede utilizarse de forma combinada para propagar efectos a todos los objetos que cumplan una condición.

```
(forall (?c - bricks)
  (when (on-site ?c ?s)
    (material-used ?c)
  )
)
```

“Para todo objeto *?c*, de tipo ladrillo (*bricks*), si está colocado en el sitio *?s*, entonces dicho ladrillo está siendo usado.”



# PDDL Avanzado (V)

## Precondiciones condicionales

- Similar a “when”, pero para precondiciones.

```
(forall (?c - bricks)
```

```
(imply (on-site ?c ?s)
```

```
(material-used ?c)
```

```
)  
)
```

Si una acción tiene esta precondición, entonces solo se podrá ejecutar si, para todos los ladrillos ?c que están en el sitio ?s, también se cumple que están siendo usados como materiales.

“Todos los ladrillos que están en el sitio ?s deben estar marcados como usados.”

○

“No puede haber ningún ladrillo en ?s que no esté usado.”

# PDDL Avanzado (VI)

## Constantes

- Son un tipo especial de objeto.
  - Se declaran en el dominio.
  - Una acción SIEMPRE puede acceder a una constante sin necesidad de que sea pasada como argumento. Es como una especie de “variable global” o “instancia siempre accesible”.

```
(define (domain <domain name>)
  (:requirements <requirements spec>)
  (:types <types spec>)
  (:constants
    norte sur este oeste - orientacion
  )
  (:predicates <predicates spec>)
```

## PDDL Numérico (I)

### Operaciones aritméticas

- A partir de la versión 2.1, PDDL es capaz de trabajar con predicados numéricos.
  - Nota: no todos los *planners* tienen la misma capacidad para operar con información numérica.
- Al ser invocados, estos predicados devuelven un número en vez de Verdadero o Falso.
  - Permite definir efectos para gestionar variables numéricas.
  - Permite definir precondiciones sobre las mismas ( $>$ ,  $<$ ,  $!=$ ).
- Requirement :**fluents**      ← Fast Forward
- Requirement :**action-costs**      ← Fast Downward

Usada exclusivamente para modelar costes de acción. FD no soporta planificación numérica en general. Solo puede representar cuánto cuesta ejecutar acciones y cuánto cuesta el plan completo, no otras magnitudes numéricas del dominio.

# PDDL Numérico (II)

```
(define (domain <domain name>)
  (:requirements <requirements spec>)
  (:types <types spec>)
  (:predicates <predicates spec>)
  (:functions <functions spec>)
  (:action <action spec>)
  (:action <action spec>)
  ...
)
```

Funciones

```
(:functions
  (distancia ?x ?y - loc)
)
```

## PDDL Numérico (III)

PDDL 2.1 permite la definición de expresiones aritméticas o relacionales para las precondiciones y efectos de las acciones.

- Usando el formato LISP (*notación prefija*):

(operador (operando1) (operando2))

```
(>
  5
  10
)
```

Diagram illustrating the LISP notation for a numeric expression. Blue arrows point from the labels to the corresponding parts of the expression: "Operador" points to ">", "Operando1" points to "5", and "Operando2" points to "10".

Los operandos pueden ser números

```
(>
  5
  (distancia ?x ?y)
)
```

Diagram illustrating the LISP notation for a numeric expression. Blue arrows point from the labels to the corresponding parts of the expression: "Operador" points to ">", "Operando1" points to "5", and "Operando2" points to "(distancia ?x ?y)".

Los operandos pueden ser funciones

```
(>
  5
  (+
    6
    (coste ?x)
  )
)
```

Diagram illustrating the LISP notation for a numeric expression. Blue arrows point from the labels to the corresponding parts of the expression: "Operador" points to ">", "Operando1" points to "5", and "Operando2" points to the entire sub-expression "(+ 6 (coste ?x))".

Los operandos pueden ser otras expresiones

## PDDL Numérico (IV)

PDDL 2.1 permite la definición de expresiones aritméticas o relacionales para las precondiciones y efectos de las acciones.

- Usando el formato LISP (*notación prefija*):  
(operador (operando1) (operando2))

```
(>
  5
  10
)
```

| Operador Relacional | Operador Aritmético |
|---------------------|---------------------|
| =                   | +                   |
| !=                  | -                   |
| >                   | *                   |
| <                   | /                   |
| >=                  |                     |
| <=                  |                     |

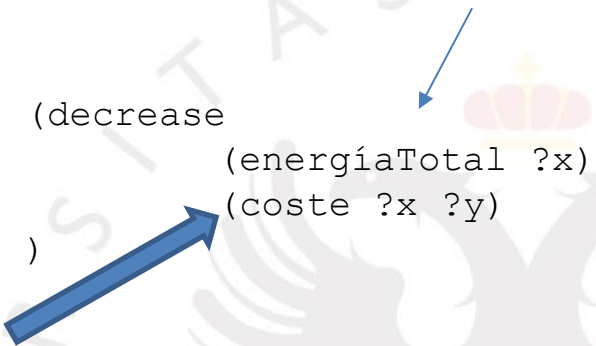
# PDDL Numérico (V)

Existen operadores predefinidos para modificar las funciones:

- (assign (funcion) (valor))
- (increase (funcion) (valor))
- (decrease (funcion) (valor))

“Decrementa la energía total del objeto x con el coste de ir de x a y”

```
(decrease
  (energíaTotal ?x)
  (coste ?x ?y)
)
```



El valor puede ser un número constante, otra función o una operación aritmética



## PDDL Numérico (VI)

En la definición del problema, el estado inicial de las funciones se instancia a un valor concreto.

```
(= (tienePlatanos mono1) 0)
(= (distancia loc1 loc2) 10)
```

La *hipótesis del mundo cerrado* no asume nada en relación a valores numéricos, es decir, no asigna por defecto ningún valor. **Todos los valores de las funciones hay que asignarlos inicialmente, incluido el 0.**

## PDDL Numérico (VII)

PDDL permite definir métricas numéricas a optimizar.

- Por defecto, un planificador (enfocado a optimización, con heurística admisible) intenta minimizar el número de acciones de un plan.
- Si definimos una métrica, esta forzará al planificador a explorar nuevas soluciones para cumplir con el requisito.



```
(:init <initial facts spec>
(:goal <goal spec>)
(:metric minimize (operación aritmética))
```

La métrica debe ir después del goal!



NOTA:

- Demostrar que un plan es óptimo es una tarea computacionalmente muy costosa.
- Por esta razón, muchos planificadores "se conforman" con aproximar (y no demostrar) un plan suficientemente "bueno" (es decir, "cercano" al óptimo).

# Índice

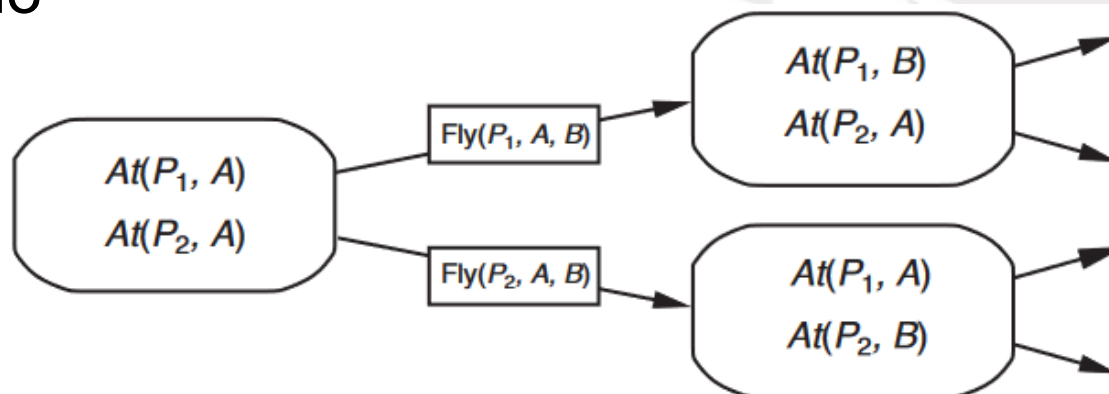
1. ¿Qué es la Planificación?
  - i. Problema de ejemplo
  - ii. Acciones
  - iii. Representación del mundo
2. Planning Domain Definition Language (PDDL)
  - i. Ontología PDDL
  - ii. Acciones PDDL
  - iii. PDDL avanzado
3. **Planificador Fast Downward**
4. Otras consideraciones
5. Práctica 3
6. Preguntas frecuentes

# Planificador FD (Fast Downward)

- FD:
  - Malte Helmert. “The Fast Downward Planning System”. *Journal of Artificial Intelligence Research* 26, pp. 191-246. 2006.
  - Malte Helmert. “Concise finite-domain representations for PDDL planning tasks”. *Artificial Intelligence* 173, pp. 503-535. 2009.
  - Es uno de los más usados, robustos y referenciados en la actualidad.
    - No obstante, debéis ser conscientes de que existen otros *planners* (como [Fast Forward](#)).
- Implementa un algoritmo de búsqueda heurística (*heuristic forward search*) para estudiar el espacio de posibles estados del mundo
  - Genera sucesores a partir del estado inicial aplicando operadores (acciones) permitidos en cada estado hasta llegar al objetivo.
    - Una acción se permite si todas sus precondiciones son satisfechas por el estado.
    - Sucesores: Estados posteriores de cada acción.
  - Condición de parada
    - Todos los predicados del objetivo están incluidos en el estado actual.

# Planificador FD (Fast Downward)

- FD:
  - Malte Helmert. “The Fast Downward Planning System”. *Journal of Artificial Intelligence Research* 26, pp. 191-246. 2006.
  - Malte Helmert. “Concise finite-domain representations for PDDL planning tasks”. *Artificial Intelligence* 173, pp. 503-535. 2009.
  - Es uno de los más usados, robustos y referenciados en la actualidad.
    - No obstante, debéis ser conscientes de que existen otros *planners* (como [Fast Forward](#)).
- Implementa un algoritmo de búsqueda heurística (*heuristic forward search*) para estudiar el espacio de posibles estados del mundo



# Planificador FD (Fast Downward)

Para el desarrollo de esta práctica usaremos el planificador FD (versión 24.06.1), que se puede descargar de:

<https://www.fast-downward.org/latest/files/release24.06/fast-downward-24.06.1.tar.gz>

Para **instalar** y **ejecutar**, véase <https://www.fast-downward.org/latest/releases/24.06/#downloads>:

```
tar -xvzf fast-downward-24.06.1.tar.gz
```

```
cd fast-downward-24.06.1
```

```
./build.py
```

```
./fast-downward.py --alias seq-opt-lmcut /path/to/domain.pddl  
/path/to/problem.pddl
```

# Planificador FD (Fast Downward)

Os recomendamos emplear Linux.

- En ocasiones hay problemas para encontrar *planners* que funcionen en Windows, pero si es para Linux suele haber más donde elegir.
- Una herramienta muy útil para emplear Linux desde Windows es [Windows Subsystem for Linux \(WSL\)](#).
  - Si usáis WSL, acordaos de hacer:
 

```
sudo apt update
```

```
sudo apt install build-essential
```

```
sudo apt install cmake
```
  - Esto instalará g++, gcc, make, y otras herramientas necesarias para compilar con cmake.



# Planificador FD (Fast Downward)

```
./fast-downward.py --alias seq-opt-lmcut
/path/to/domain.pddl
/path/to/problem.pddl
```

```
stack b5 b3 (1)
unstack b2 b4 (1)
put_down b2 (1)
unstack b5 b3 (1)
stack b5 b2 (1)
unstack b3 b1 (1)
stack b3 b5 (1)
[t=0.003768s, 10228 KB] Plan length: 7 step(s).
[t=0.003768s, 10228 KB] Plan cost: 7
[t=0.003768s, 10228 KB] Expanded 11 state(s).
[t=0.003768s, 10228 KB] Reopened 0 state(s).
[t=0.003768s, 10228 KB] Evaluated 21 state(s).
[t=0.003768s, 10228 KB] Evaluations: 21
[t=0.003768s, 10228 KB] Generated 29 state(s).
[t=0.003768s, 10228 KB] Dead ends: 0 state(s).
[t=0.003768s, 10228 KB] Expanded until last jump: 3 state(s).
[t=0.003768s, 10228 KB] Reopened until last jump: 0 state(s).
[t=0.003768s, 10228 KB] Evaluated until last jump: 7 state(s).
[t=0.003768s, 10228 KB] Generated until last jump: 8 state(s).
[t=0.003768s, 10228 KB] Number of registered states: 21
[t=0.003768s, 10228 KB] Int hash set load factor: 21/32 = 0.656250
[t=0.003768s, 10228 KB] Int hash set resizes: 5
[t=0.003768s, 10228 KB] Search time: 0.000617s
[t=0.003768s, 10228 KB] Total time: 0.003768s
Solution found.
Peak memory: 10228 KB
Remove intermediate file output.sas
search exit code: 0
INFO Planner time: 0.07s
```

Busca optimalidad en número de acciones.

```
./fast-downward.py --alias lama-first
/path/to/domain.pddl
/path/to/problem.pddl
```

```
put_down b5 (1)
unstack b2 b4 (1)
put_down b2 (1)
unstack b3 b1 (1)
put_down b3 (1)
pick_up b5 (1)
stack b5 b2 (1)
pick_up b3 (1)
stack b3 b5 (1)
[t=0.004742s, 10364 KB] Plan length: 9 step(s).
[t=0.004742s, 10364 KB] Plan cost: 9
[t=0.004742s, 10364 KB] Expanded 16 state(s).
[t=0.004742s, 10364 KB] Reopened 0 state(s).
[t=0.004742s, 10364 KB] Evaluated 17 state(s).
[t=0.004742s, 10364 KB] Evaluations: 34
[t=0.004742s, 10364 KB] Generated 59 state(s).
[t=0.004742s, 10364 KB] Dead ends: 0 state(s).
[t=0.004742s, 10364 KB] Number of registered states: 17
[t=0.004742s, 10364 KB] Int hash set load factor: 17/32 = 0.531250
[t=0.004742s, 10364 KB] Int hash set resizes: 5
[t=0.004742s, 10364 KB] Search time: 0.000659s
[t=0.004742s, 10364 KB] Total time: 0.004742s
Solution found.
Peak memory: 10364 KB
Remove intermediate file output.sas
search exit code: 0
INFO Planner time: 0.07s
```

Busca encontrar rápidamente planes factibles (no necesariamente óptimos).

## Planificador FD (Fast Downward)

Veréis que la salida de FD es muy verbosa. Si queréis aislar con facilidad el plan obtenido podéis hacer:

```
./fast-downward.py --alias seq-opt-lmcut --plan-file plan_resultante.txt
dominio1.pddl problema1.pddl
```

Esto permitiría guardar automáticamente el plan en el fichero indicado (en este ejemplo, *plan\_resultante.txt*).

Por defecto, aunque no se use *--plan-file*, el plan se guarda en un fichero llamado *sas\_plan*.

## Planificador FD (Fast Downward)

De cara a **depurar vuestro código** posiblemente os interese generar el fichero intermedio *output.sas*:

```
./fast-downward.py --alias seq-opt-lmcut --translate dominio1.pddl  
problema1.pddl
```

El fichero *output.sas* contiene el problema PDDL traducido a una representación SAS+ finita y explícita, lista para la búsqueda. Incluye las variables de estado y sus valores posibles, el estado inicial, el objetivo, y las acciones ya instanciadas con sus precondiciones, efectos y coste.

Véase <https://www.fast-downward.org/latest/documentation/translator-output-format/>

## Planificador FD (Fast Downward)

En caso de obtener algún error relativo al uso de *axioms* (*derived predicates*), se podría emplear:

```
./fast-downward.py --alias seq-sat-lama-2011 --search-time-limit 1s
dominio1.pddl problema1.pddl
```

Con el **search-time-limit** que sea suficiente y razonable.

Ejemplo de predicado derivado o axioma:

```
(:derived (conectado ?x ?y)
  (or (camino ?x ?y)
    (exists (?z) (and (camino ?x ?z) (conectado ?z ?y)))))
```

Definición lógica de cuándo “conectado” debe considerarse cierto. Se trata de un predicado cuya verdad (True/False) no cambia directamente (por medio de los efectos de las acciones) sino que es inferida a partir de otros predicados/hechos.

**Nota importante:** en cada ejercicio de la práctica debéis emplear exactamente el alias que se indique en el enunciado. No podéis escoger vosotros cuál emplear. Es decir, debéis ser capaces de implementar vuestros códigos de modo que se ejecuten correctamente con el alias solicitado.

## Planificador FD (Fast Downward)

Es muy importante que reviséis con calma la documentación del *planner*, para familiarizaros con las particularidades del mismo. Por ejemplo:

- FD no permite efectos condicionales (*when*) en combinación con información numérica (*increase*).  
(**when** <condición> (**increase** (**total-cost**) 3))  
no está soportado en FD.
- En FD, no se admiten funciones numéricas arbitrarias. La única función numérica soportada es **total-cost**, usada exclusivamente para modelar costes de acción.

<https://www.fast-downward.org/24.06/documentation/pddl-support/>  
(sección *Limitations*)

# Índice

1. ¿Qué es la Planificación?
  - i. Problema de ejemplo
  - ii. Acciones
  - iii. Representación del mundo
2. Planning Domain Definition Language (PDDL)
  - i. Ontología PDDL
  - ii. Acciones PDDL
  - iii. PDDL avanzado
3. Planificador Fast Downward
4. **Otras consideraciones**
5. Práctica 3
6. Preguntas frecuentes





# Editores

Los ficheros de dominio y problema pueden ser editados con:

1. Un editor de texto cualquiera (Notepad++, Gedit, Atom, Sublime Text)
2. PDDL Online Editor (<http://editor.planning.domains/>)
  - Permite ejecutar los dominios según se construyen.
  - Adecuado para ejercitarse online con problemas de prueba y, luego, usar FD para resolver los problemas de la práctica.
  - El editor online tiene autoguardado. Pero es recomendable no abandonar la página antes de guardar localmente todo el trabajo hecho.

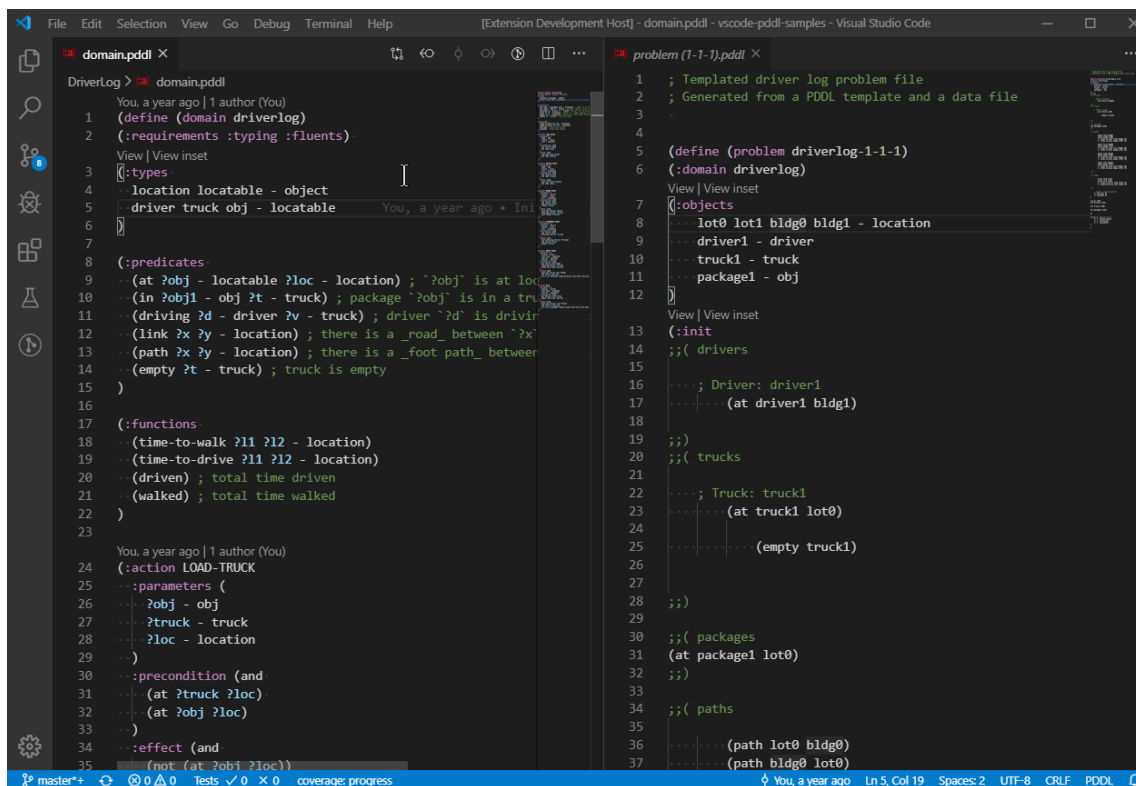


# Editores

Los ficheros de dominio y problema pueden ser editados con:

## 3. Visual Studio tiene una extensión para PDDL

— <https://marketplace.visualstudio.com/items?itemName=jan-dolejsi.pddl>



```

domain.pddl
1 You, a year ago | 1 author (You)
2 (define (domain driverlog)
3   (:requirements :typing :fluents)
4   View | View inset
5   (:types
6     --location locatable - object
7     --driver truck obj - locatable
8   )
9   (:predicates
10    --(at ?obj - locatable ?loc - location) ; ?obj is at loc
11    --(in ?obj1 - obj ?t - truck) ; package ?obj1 is in a truck
12    --(driving ?d - driver ?v - truck) ; driver ?d is driving
13    --(link ?x ?y - location) ; there is a _road_between_ ?x
14    --(path ?x ?y - location) ; there is a _foot path_between_
15    --(empty ?t - truck) ; truck is empty
16   )
17   (:functions
18    --(time-to-walk ?l1 ?l2 - location)
19    --(time-to-drive ?l1 ?l2 - location)
20    --(driven) ; total time driven
21    --(walked) ; total time walked
22   )
23   You, a year ago | 1 author (You)
24   (:action LOAD-TRUCK
25     :parameters (
26       --?obj - obj
27       --?truck - truck
28       --?loc - location
29     )
30     :precondition (and
31       --(at ?truck ?loc)
32       --(at ?obj ?loc)
33     )
34     :effect (and
35       --(not (at ?obj ?loc))
36     )
37   )
38 )

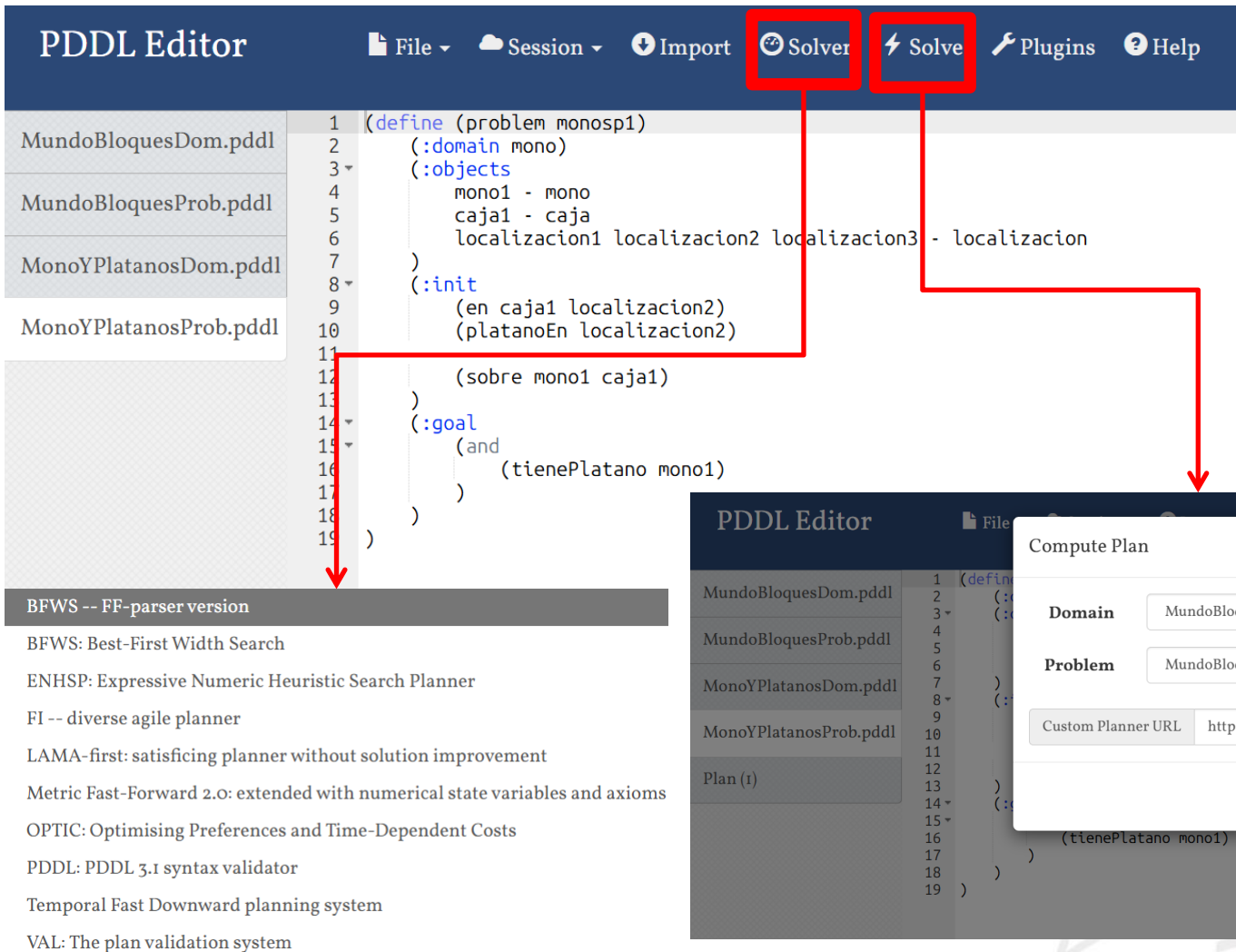
problem(1-1-1).pddl
1 ; Templated driver log problem file
2 ; Generated from a PDDL template and a data file
3
4
5 (define (problem driverlog-1-1-1)
6   (:domain driverlog)
7   View | View inset
8   (:objects
9     lot0 lot1 bldg0 bldg1 - location
10    driver1 - driver
11    truck1 - truck
12    package1 - obj
13  )
14  View | View inset
15  (:init
16    ;;; drivers
17    ;;;
18    ;;; Driver: driver1
19    ;;; (at driver1 bldg1)
20    ;;;
21    ;;; trucks
22    ;;;
23    ;;; Truck: truck1
24    ;;; (at truck1 lot0)
25    ;;; (empty truck1)
26    ;;;
27    ;;;
28    ;;;
29    ;;; packages
30    ;;; (at package1 lot0)
31    ;;;
32    ;;;
33    ;;; paths
34    ;;; (path lot0 bldg0)
35    ;;; (path bldg0 lot0)
36  )
37 )
  
```

Permite:

- Editar ficheros PDDL
- Ejecutar el planificador
- Autocompletado
- Detección de errores sintácticos
- Autofix de warnings
- Visualización de modelos
- Visualización de planes
- ...

# PDDL Online Editor

(<http://editor.planning.domains/>)



**PDDL Editor**

File Session Import Solver Solve Plugins Help

MundoBloquesDom.pddl  
MundoBloquesProb.pddl  
MonoYPlatanosDom.pddl  
MonoYPlatanosProb.pddl

```

1 (define (problem monosp1)
2   (:domain mono)
3   (:objects
4     mono1 - mono
5     caja1 - caja
6     localizacion1 localizacion2 localizacion3 - localizacion
7   )
8   (:init
9     (en caja1 localizacion2)
10    (platanoEn localizacion2)
11  )
12  (:goal
13    (and
14      (tienePlatano mono1)
15    )
16  )
17 )
18 )
19 )

```

BFWS -- FF-parser version  
BFWS: Best-First Width Search  
ENHSP: Expressive Numeric Heuristic Search Planner  
FI -- diverse agile planner  
LAMA-first: satisficing planner without solution improvement  
Metric Fast-Forward 2.0: extended with numerical state variables and axioms  
OPTIC: Optimising Preferences and Time-Dependent Costs  
PDDL: PDDL 3.1 syntax validator  
Temporal Fast Downward planning system  
VAL: The plan validation system

**Compute Plan**

Domain: MundoBloquesDom.pddl  
Problem: MundoBloquesProb.pddl  
Custom Planner URL: http://solver.planning.domains  
Plan

# Cómo depurar en PDDL

Depurar código PDDL no es especialmente fácil:

- Muchos de los mensajes de error que proporciona el *planner* no son particularmente informativos... En este sentido, se recomienda revisar el material de apoyo disponible en PRADO.

## • Consejo general:

- Simplificad vuestro problema al máximo e id depurando con dominios progresivamente más complejos:
  1. Definid un dominio con una sola acción.
  2. Definid un problema cuyo plan sea ejecutar esa acción.
  3. Lanzad el planificador y comprobad que el resultado sea correcto.
  4. incluid una nueva acción al dominio.
  5. Modificad el problema para que el plan contenga la nueva acción.
  6. Lanzad el planificador y comprobad que el resultado sea correcto.
  7. Repetid 4 – 6 hasta terminar.

## Primeros pasos

1. Revisar con calma los ejemplos (mundo de bloques y mono/plátanos) proporcionados en [http://editor.planning.domains/#read\\_session=wQdLTQZvUN](http://editor.planning.domains/#read_session=wQdLTQZvUN)
2. Con respecto al problema del mono y los plátanos:
  - I. Modificad la localización de los plátanos.
    - ¿Es correcto el resultado del planificador? Arregladlo si no lo es.
  - II. Incluid una nueva acción para que el mono empuje la caja.
    - El mono debe estar en la misma posición que la caja.
    - Cuando la empuja el mono debe moverse con ella.
  - III. Incluid una nueva acción para que el mono suba a la caja.
    - El mono debe estar en la misma posición que la caja.
    - Y no debe estar ya encima de ella.
  - IV. Definid una acción para que el mono sea capaz de moverse por el mundo sin necesidad de empujar la caja.

# Índice

1. ¿Qué es la Planificación?
  - i. Problema de ejemplo
  - ii. Acciones
  - iii. Representación del mundo
2. Planning Domain Definition Language (PDDL)
  - i. Ontología PDDL
  - ii. Acciones PDDL
  - iii. PDDL avanzado
3. Planificador Fast Downward
4. Otras consideraciones
5. **Práctica 3**
6. Preguntas frecuentes

- La práctica consiste en resolver 5 ejercicios de planificación (ver enunciado de la práctica):
  - **La puntuación de cada ejercicio varía entre 1 y 3 ptos.**
  - Los ejercicios con **errores sintácticos califican con 0 ptos**, independientemente de la documentación entregada. **El código debe estar muy bien comentado.**
  - Se entrega **memoria (10 págs. como máximo)**:
    - En cada ejercicio se deben explicar los principios de diseño empleados a la hora de implementar cada problema y cada dominio, así como las dificultades encontradas durante su desarrollo, y el uso realizado de herramientas de IA generativa.
    - Se deben responder las preguntas conceptuales y/o teóricas planteadas, en caso de haberlas.
- La entrega se realizará a través de PRADO, y consistirá en un **fichero ZIP** que contenga **los ficheros escritos en PDDL para cada ejercicio** (dominio y problema), y el **fichero PDF** con la memoria anteriormente mencionada.

**Fecha de entrega: 02 de Junio de 2026 23:59**

# Índice

1. ¿Qué es la Planificación?
  - i. Problema de ejemplo
  - ii. Acciones
  - iii. Representación del mundo
2. Planning Domain Definition Language (PDDL)
  - i. Ontología PDDL
  - ii. Acciones PDDL
  - iii. PDDL avanzado
3. Planificador Fast Downward
4. Otras consideraciones
5. Práctica 3
6. **Preguntas frecuentes**



## Consideraciones generales y preguntas frecuentes

### 1. Si tenemos un objeto, ¿este puede tener atributos?

- Supongamos que tenemos un objeto de tipo persona que se llama Pablo. Si queremos que ese objeto tenga información extra (por ejemplo, el nombre de su padre) deberíamos usar un predicado lógico (padreDe Pablo Guillermo). Si queremos, podríamos determinar su peso con  $(= (\text{peso Pablo}) 65)$  usando funciones. Así es como se puede añadir información extra a un objeto.

### 2. ¿Cómo es posible definir un *grid* en PDDL?

- De nuevo, empleando lógica de predicados. Si tenemos un grid de 2x2, y las localizaciones (A, B, C, D) están conectadas entre ellas si son adyacentes (horizontal y verticalmente, pero no diagonalmente), se podría representar así con lógica de predicados en el fichero del problema:

|   |   |
|---|---|
| A | B |
| C | D |

(conectado A B)  
(conectado A C)  
(conectado B A)  
(conectado B D)  
(conectado C A)  
(conectado C D)  
(conectado D B)  
(conectado D C)

## Consideraciones generales y preguntas frecuentes

3. Ejecuto mi dominio y mi problema, no obtengo ningún error claro, pero tampoco se genera ningún plan. ¿Qué puede estar pasando?
  - Este comportamiento, por ejemplo, en Metric-FF suele ir asociado al mensaje *predicate XXXXXX is declared to use unknown or empty type YYYYYY*. Generalmente, se corresponde a no incluir un espacio entre el objeto definido y su tipo, por ejemplo: *A1 A2 A3- Unidad* en lugar de *A1 A2 A3 - Unidad*
  - En FD, podríais obtener, sencillamente, *Search stopped without finding a solution*.
4. Mi código tarda mucho en ejecutarse. ¿A qué puede deberse?
  - Evidentemente, este comportamiento puede estar asociado con muchas causas. Pero una de las principales es no acotar adecuadamente las precondiciones de las acciones. Otro motivo puede encontrarse en hacer un uso inadecuado de *forall* y *exist*.

# Consideraciones generales y preguntas frecuentes

## 4. Mi código tarda mucho en ejecutarse. ¿A qué puede deberse?

- Objetivos “redundantes” también permiten optimizar el tiempo de resolución y el número de estados evaluados:

0.24 segundos, evaluando 7261 estados

```
(exists (?edi - edificio)
  (and (edificioEsTipo ?edi Hospital)
    (posicionadoEn ?edi LOC23)
  )
)
```

0.18 segundos, evaluando 5999 estados

```
(exists (?edi - edificio)
  (and (edificioEsTipo ?edi Hospital)
    (construido ?edi)
    (posicionadoEn ?edi LOC23)
  )
)
```

- Definición adecuada de predicados. Ejemplo del curso pasado:

- Pasar de tener 5 predicados de tipo (*estar-en* <personaje> <localizacion>) a tener un único predicado (*comunidad-en* <localizacion>) cuando se forma y viaja la comunidad (ya que unos no pueden ir sin los otros).
  - Esto hace que pase de más de 700000 estados evaluados y más de 2.5 millones de estados generados (entre 400-500 segundos de ejecución) a 3059 evaluados y 6957 generados (0.8 segundos).

## Consideraciones generales y preguntas frecuentes

### 5. No acabo de ver la utilidad de las constantes. ¿Eso cómo va?

- Las constantes nos permiten declarar objetos que están presentes en todas las instancias de un problema.

- Por ejemplo, en un determinado contexto podríamos tener distintos tipos de coche:

```
(:constants
    Formula1 SUV Microcar - tipoCoche
)
```

- Cuya conducción podría variar de uno a otro y, para ello, podríamos definir un predicado que nos permita identificar el tipo de coche:

```
(cocheEs ?c - Coche ?tC - tipoCoche)
```

- A continuación, operar con él en las acciones correspondientes:

```
:precondition
    (and
        (cocheEs ?c Formula1)
```

- Y definirlos en el fichero de problema

```
(:objects
    F1 F2 Microcar1 - Coche
    C1 C2 - Conductor
)
```

```
(:init
    (cocheEs F1 Formula1)
    (cocheEs F2 Formula1)
    (cocheEs Microcar1 Microcar)
)
```

## Consideraciones generales y preguntas frecuentes

6. He definido tipos y constantes, pero quiero emplear una constante para definir un tipo y no me deja. ¿Cómo puedo hacer?

- Una constante no es un tipo y, como consecuencia, no se puede emplear como tal.
- Resulta clave diseñar una adecuada jerarquía de tipos, suficientemente clara, rica y semánticamente consistente.
- En el ejemplo de la slide anterior, una cosa es un coche y otra un tipo de coche. Semánticamente no es lo mismo. Se podría definir, en el fichero de dominio, una jerarquía de tipos como la siguiente:

```
(:types
  Entidad tipoConductor tipoCoche - object
  Coche Conductor - Entidad
)
```

- En la P2, si teníais problemas definiendo restricciones, posiblemente se debía a una inadecuada codificación de variables. Del mismo modo, en esta práctica, si encontráis dificultades a la hora de definir las acciones, muy probablemente se trate de algún problema en vuestra jerarquía de tipos o en los predicados definidos.

# Consideraciones generales y preguntas frecuentes

7. ¿Cuándo es necesario introducir un *and* en el *goal*? ¿Y en las acciones?
- Si solo hay un predicado, no es necesario. Puede incluirse o puede no incluirse. No afecta. Esto es aplicable tanto al *:goal* incluido en el fichero de problema como a las acciones del fichero de dominio.

```
(:action irA
  :parameters (?m - mono ?x ?y - loc)
  :precondition
    (and
      (en ?m ?x)
    )
  :effect
    (and
      (en ?m ?y)
      (not (en ?m ?x))
    )
)
```



```
(:action irA
  :parameters (?m - mono ?x ?y - loc)
  :precondition
    (en ?m ?x)
  :effect
    (and
      (en ?m ?y)
      (not (en ?m ?x))
    )
)
```

```
(:goal
  (and
    (tienePlatano mono1)
  )
)
```



```
(:goal
  (tienePlatano mono1)
)
```



# Consideraciones generales y preguntas frecuentes

## 7. ¿Cuándo es necesario introducir un *and* en el *goal*? ¿Y en las acciones?

- Si hay más de uno, es necesario incluirlo. De lo contrario, se obtiene un *syntax error*.

```
(:action cogerPlatanos
  :parameters (?m - mono ?c - caja ?x - loc)
  :precondition
    (and
      (sobre ?m ?c)
      (en ?c ?x)
      (platanoEn ?x)
    )
  :effect
    (and
      (tienePlatano ?m)
    )
)
```

```
(:action cogerPlatanos
  :parameters (?m - mono ?c - caja ?x - loc)
  :precondition
    (sobre ?m ?c)
    (en ?c ?x)
    (platanoEn ?x)
  :effect
    (and
      (tienePlatano ?m)
    )
)
```

Error de Metric-FF

ff: parsing domain file  
mono\_dominio.pddl: syntax error in line 23, '(':  
domain definition expected

Error de FD

Effect tag is expected to be 'effect'  
Syntax: (:action NAME [:parameters PARAMETERS]? [:precondition PRECONDITION]? :effect EFFECT)  
Got: ['en', '?c', '?x']  
translate exit code: 31



## Consideraciones generales y preguntas frecuentes

8. ¿Puedo crear un objeto mineral, ubicado en una determinada localización, y poner como objetivo que se extraiga ese mineral concreto?

- No. Mineral es una constante definida en el fichero de dominio. Y el planificador debe escoger automáticamente el nodo de minerales más cercano (*metric is plan length*).

```
(depositoEn mineral1 loc24)
(depositoEn mineral2 loc44)
)
(:goal
  (generando mineral1)
)
```

VS

```
(depositoEn Mineral loc24)
(depositoEn Mineral loc44)
)
(:goal
  (generando Mineral)
)
```



## Consideraciones generales y preguntas frecuentes

9. ¿Qué hace que una tarea de planificación automática sea más difícil que otra?

– **Subset de PDDL:**

- Si queremos/necesitamos operar con información numérica, temporal (acciones solapadas), incierta,... A mayor expresividad de nuestro lenguaje, mayor complejidad.

– **Dominio:**

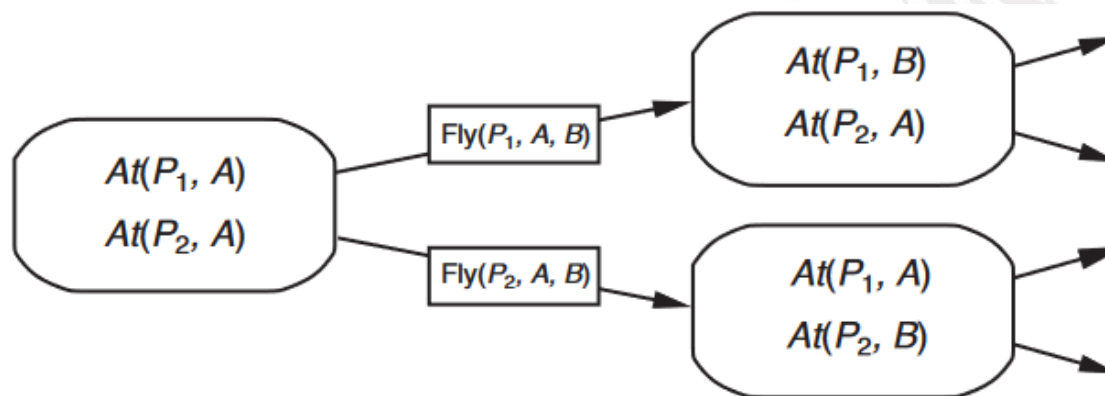
- Número de acciones y número de parámetros de cada una.
- Cantidad de predicados y número de parámetros.

– **Problema:**

- Cantidad de objetos.

# Consideraciones generales y preguntas frecuentes

9. ¿Qué hace que una tarea de planificación automática sea más difícil que otra?
- Se puede tener un dominio sencillo, pero definir un problema complicado y no poder resolverlo.
    - Ejemplo: el mundo de bloques con cientos o miles de bloques.
  - Recordemos que, por debajo, tenemos un proceso de búsqueda, en donde se van instanciando acciones (convertimos variables en objetos).
    - Si tenemos una acción con 4 parámetros, y 100 posibles objetos para cada uno, tendríamos  $100^4$  posibles acciones a instanciar.



Stuart Russell and Peter Norvig. "Artificial Intelligence: A Modern Approach", 4th, Global ed. (2022). Página 367



UNIVERSIDAD  
DE GRANADA



# Técnicas de los Sistemas Inteligentes

Grado en Ingeniería Informática

**Curso 2025-26. Seminario 3**

**Planificación Clásica (PDDL) y Práctica 3**

Pablo Mesejo Santiago y David Criado Ramón

**Departamento de Ciencias de la  
Computación e Inteligencia Artificial**

<http://decsai.ugr.es>